

C++ Template 延伸自主練習作業

學生成績管理系統

主題：STL 與基礎資料結構入門 — 第 2 週整合應用小專題

學 號	_____
姓 名	_____
GitHub	https://github.com/your-account/your-repository
編譯環境	g++ -std=c++17 main.cpp -o main
繳交檔案	main.cpp、README.md、report.pdf

※ 繳交前請將 GitHub 網址改成自己的 Public repository，並確認可在未登入狀態開啟。

一、自主學習主題與目標

本次自主學習的主題是 **C++ 標準模板函式庫 (STL, Standard Template Library) 與基礎資料結構**。它接續課堂 Chapter 16 所教的 function template 與 class template 觀念，把「同一份程式碼可以套用到不同型別」的想法，延伸到實務上每天都會用到的 STL 容器與演算法。

在學習這個主題之前，我寫程式大多還停留在使用原生陣列（如 `int arr[100]`）來儲存資料，常常要自己處理「陣列開多大」「資料超過容量怎麼辦」「排序要自己寫氣泡排序」這類問題。透過這次自主學習，我希望達成以下幾個目標：

1. **理解 template 與 STL 之間的關係**：弄清楚為什麼 `vector`、`map` 這些容器可以裝任何型別，它們背後其實就是 class template 的實際應用。
2. **熟悉常見 STL 容器的用途**：包含 `vector`、`stack`、`queue`、`priority_queue`、`set`、`map`，知道每一種大致對應到哪一種資料結構。
3. **能依問題需求選擇合適的資料結構**：不再「什麼都用陣列」，而是依照需要的操作（要不要排序、要不要快速查詢、要不要先進先出）來選容器。
4. **能寫出可在 `g++ -std=c++17` 下編譯執行的完整程式**：以「學生成績管理系統」整合 `struct`、`vector`、`sort`、查詢與統計等功能。
5. **為大二的資料結構課程打下先備基礎**：先用 STL「會用」這些資料結構，之後在資料結構課就能理解它們「怎麼實作出來」。

整體來說，這次學習讓我從「會用 template 語法」進步到「理解 template 在整個 C++ 標準函式庫裡扮演的角色」，並且能實際做出一個有完整功能的小程式。

二、Template 與 STL 的關係

2-1 從 function template 出發

課堂上學到的 function template，最大的好處是「一份程式碼，多種型別共用」。例如本作業的固定挑戰題：

```
template <class T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
}
```

這個 `getMax` 不只能比較 `int`，也能比較 `double`、`char`，甚至只要型別有定義 `>` 運算子的都可以。編譯器會在「真正被呼叫時」依照傳入的型別，自動幫我們「長出」對應的版本，這個過程稱為 **樣板實體化 (template instantiation)**。也就是說，當我在統計功能裡寫 `getMax(maxScore, s.score)`（兩個都是 `int`）時，編譯器其實是幫我產生了一個 `int getMax(int, int)` 的版本。

2-2 從 function template 到 class template

class template 把同樣的概念套用到「類別」上。STL 的容器就是最典型的 class template。例如：

```
vector<int>    v1;    // 裝 int 的動態陣列
vector<string> v2;    // 裝 string 的動態陣列
vector<Student> v3;   // 裝自訂 struct 的動態陣列
```

`vector` 本身的定義大致是 `template <class T> class vector { ... };`，所以我們只要把角括號裡的 `T` 換成想要的型別，

就能得到一個專門裝那種型別的容器。這正是「STL 本質上是 **template** 的實際應用」這句話的意思。同樣的道理也適用於 `map<string, int>`、`stack<char>`、`set<int>` 等等。

2-3 STL 三大組成與 **template** 的關係

STL 大致由三個部分組成，彼此都靠 **template** 串接：

組成	說明	與 template 的關係
Containers（容器）	儲存資料的結構，如 <code>vector</code> 、 <code>map</code>	都是 <code>class template</code> ，可裝任意型別
Algorithms（演算法）	對資料做運算，如 <code>sort</code> 、 <code>find</code>	是 <code>function template</code> ，可作用於不同容器與型別
Iterators（迭代器）	連接容器與演算法的橋樑	讓 <code>algorithm</code> 不用知道容器內部細節即可走訪

舉例來說，`std::sort` 並不需要知道你給它的是裝 `int` 的 `vector` 還是裝 `Student` 的 `vector`，它只透過迭代器（`begin()`、`end()`）來操作。這種「演算法與容器解耦」的設計，正是 **template** 帶來的最大威力：一套 **sort 演算法** 可以排序任何容器、任何型別。

三、常見 STL 容器用途整理

第 1 週的基礎練習讓我把以下幾種容器都操作過一遍，下面整理它們的用途與對應的資料結構。

3-1 容器一覽表

容器	對應資料結構	特性	常見用途	主要操作
<code>vector</code>	動態陣列	可隨機存取、尾端增刪快	一般序列資料	<code>push_back</code> ， <code>[]</code> ， <code>size</code>
<code>string</code>	字元動態陣列	專門處理文字	字串處理	<code>+</code> ， <code>length</code> ， <code>substr</code>
<code>stack</code>	堆疊	後進先出 LIFO	括號配對、回溯	<code>push</code> ， <code>pop</code> ， <code>top</code>
<code>queue</code>	佇列	先進先出 FIFO	排隊模擬、BFS	<code>push</code> ， <code>pop</code> ， <code>front</code>
<code>priority_queue</code>	堆積 heap	取出時自動取最大/最小	優先處理、Top-K	<code>push</code> ， <code>pop</code> ， <code>top</code>
<code>set</code>	紅黑樹（平衡二元搜尋樹）	自動排序、不重複	去重、有序集合	<code>insert</code> ， <code>find</code> ， <code>count</code>
<code>map</code>	紅黑樹	鍵值對、依鍵排序	字典、計數、索引	<code>[]</code> ， <code>find</code> ， <code>count</code>

3-2 各容器補充說明

vector（動態陣列）：最常用的容器，可以想成「會自動長大的陣列」。支援 `[]` 隨機存取，尾端 `push_back` 平均時間複雜度 $O(1)$ 。本專題就是用 `vector<Student>` 來存所有學生。

stack（堆疊，LIFO）：第 1 週練習用它做「括號配對檢查」——遇到左括號就 `push`，遇到右括號就檢查 `top` 是否配對。它「後放的先拿」的特性正好適合處理巢狀結構。

queue（佇列，FIFO）：第 1 週練習用它模擬「排隊」——先來的先服務。`front` 看最前面的人，`pop` 服務（移除）最前面的人。

priority_queue（優先佇列）：底層是 `heap`，每次 `top` 取出的都是目前最大（預設）的元素，適合「Top-K」「每次都要拿最大值」的情境。

set（集合）：底層是平衡二元搜尋樹，元素會自動排序且不重複，常用來去除重複資料。

map（字典 / 關聯陣列）：以「鍵 → 值」儲存，鍵會自動排序。第 1 週練習用 `map<string,int>` 統計每個單字出現次數，

寫起來非常直覺：`count[word]++`。

3-3 我如何選容器

在這個專題裡，我最後選用 `vector<Student>` 作為主要儲存容器，理由是：

- 學生數量不固定，需要動態增加 → `vector` 的 `push_back` 很適合。
- 功能 3 要求用 `std::sort` 依成績排序，而 `sort` 可以直接作用在 `vector` 上；如果改用 `map`，因為 `map` 是依「鍵」排序、不能依「值」重排，反而要先轉成 `vector` 才能排，比較麻煩。
- 資料量不大，查詢用線性搜尋（逐筆比對學號）就足夠，不需要 `map` 的 $O(\log n)$ 查詢優勢。

作業也允許用 `map<string, Student>`，那種做法在「依學號查詢」會更快（ $O(\log n)$ ），但排序時要另外轉成 `vector`。兩種各有優缺點，我選擇了讓「排序」最自然的 `vector` 版本。

四、第 2 週小專題設計說明

本專題實作「學生成績管理系統」，以選單方式操作，提供六個選項（含離開）。整體設計如下。

4-1 資料設計

每位學生用一個 `struct` 表示，包含三個欄位：

```
struct Student {
    string id;    // 學號
    string name; // 姓名
    int    score; // 成績 (0~100)
};
```

所有學生存放在 `vector<Student> students;` 中。

4-2 功能規劃

功能	對應函式	設計重點
新增學生	<code>addStudent</code>	先檢查學號是否重複，重複則拒絕並提示；成績限制 0~100
列出所有學生	<code>listStudents</code>	用 <code>setw</code> 對齊欄位，空資料時提示
依成績排序	<code>sortByScore</code>	<code>std::sort + lambda</code> ，由高到低
依學號查詢	<code>searchById</code>	線性搜尋，找不到時輸出訊息
成績統計	<code>showStatistics</code>	平均、最高、最低、及格 / 不及格人數
離開	<code>main</code> 內 <code>case 0</code>	結束程式

4-3 重複學號的處理策略

作業說明允許自行決定「覆蓋、拒絕新增或提示錯誤」。我選擇 **拒絕新增並提示錯誤**，因為這樣最安全，避免使用者不小心把舊資料蓋掉。實作上是在加入前先用迴圈比對所有現有學號。

4-4 輸入防呆設計

為了讓程式更穩定，我加了一個 `readInt` 小工具，當使用者輸入非數字時不會讓程式卡死，而是清除錯誤狀態並要求重新輸入。成績也限制必須在 0~100 之間。

五、主要程式架構與重要程式片段

5-1 整體架構

程式採「主迴圈 + 功能函式」的模組化結構：

```
main()
├─ printMenu()          顯示選單
├─ readInt()            安全讀取選項
└─ switch(choice)
    ├─ addStudent()      功能 1
    ├─ listStudents()    功能 2
    ├─ sortByScore()     功能 3
    ├─ searchById()      功能 4
    └─ showStatistics()  功能 5
```

5-2 Template function（固定挑戰題）

依作業固定要求，定義 `getMax` 與 `getMin` 兩個 template function：

```
template <class T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
}

template <class T>
T getMin(T a, T b) {
    return (a < b) ? a : b;
}
```

使用位置：這兩個 template function 在「功能 5：成績統計」的 `showStatistics` 函式中被呼叫，用來找出全班的最高分與最低分（見 5-5）。

5-3 新增學生（含重複檢查）

```
void addStudent(vector<Student>& students) {
    Student s;
    cout << "請輸入學號：";
    cin >> s.id;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    // 檢查學號是否重複
    for (const Student& exist : students) {
        if (exist.id == s.id) {
            cout << "[錯誤] 學號 " << s.id << " 已存在，無法重複新增。\\n";
            return;
        }
    }
    cout << "請輸入姓名：";
    getline(cin, s.name);          // 姓名可能含空白，用 getline

    s.score = readInt("請輸入成績 (0~100)：");
}
```

```

while (s.score < 0 || s.score > 100) {
    cout << "[提示] 成績需介於 0 到 100 之間。\\n";
    s.score = readInt("請重新輸入成績 (0~100) : ");
}
students.push_back(s);
cout << "[成功] 已新增學生 : " << s.id << " " << s.name << " " << s.score << " 分\\n";
}

```

5-4 依成績排序 (sort + lambda)

```

sort(students.begin(), students.end(),
    [](const Student& a, const Student& b) {
        return a.score > b.score;    // 大的在前 → 由高到低
    });

```

這裡用 lambda 當作比較函式，告訴 sort：當 `a.score > b.score` 時，`a` 要排在 `b` 前面，因此結果會是「成績由高到低」。

5-5 輸入防呆工具 readInt

為了避免使用者輸入非數字造成程式卡死，所有「讀整數」的地方都透過這個小工具：

```

int readInt(const string& prompt) {
    int value;
    while (true) {
        cout << prompt;
        if (cin >> value) {
            cin.ignore(numeric_limits<streamsize>::max(), '\\n');
            return value;
        }
        cin.clear();    // 清除錯誤旗標
        cin.ignore(numeric_limits<streamsize>::max(), '\\n');
        cout << "輸入格式錯誤, 請重新輸入數字。\\n";
    }
}

```

5-6 列出所有學生 (欄位對齊)

```

void listStudents(const vector<Student>& students) {
    if (students.empty()) {
        cout << "[提示] 目前沒有任何學生資料。\\n";
        return;
    }
    cout << left << setw(12) << "學號"
        << setw(14) << "姓名"
        << right << setw(6) << "成績" << "\\n";
    for (const Student& s : students) {
        cout << left << setw(12) << s.id
            << setw(14) << s.name
            << right << setw(6) << s.score << "\\n";
    }
    cout << "共 " << students.size() << " 位學生。\\n";
}

```

這裡用 `setw` 搭配 `left` / `right` 讓欄位對齊，讓輸出格式清楚易讀。

5-7 依學號查詢（線性搜尋）

```
void searchById(const vector<Student>& students) {
    string target;
    cout << "請輸入要查詢的學號：";
    cin >> target;
    for (const Student& s : students) {
        if (s.id == target) {
            cout << "[找到] 學號：" << s.id
                << " 姓名：" << s.name
                << " 成績：" << s.score << " 分\n";
            return;
        }
    }
    cout << "[查無此人] 找不到學號為 " << target << " 的學生。\\n";
}
```

5-8 成績統計（呼叫 template function）

```
int maxScore = students[0].score;    // 初始值設為第一位學生
int minScore = students[0].score;
int passCount = 0, failCount = 0, total = 0;

for (const Student& s : students) {
    total += s.score;
    maxScore = getMax(maxScore, s.score);    // ← 呼叫 template function
    minScore = getMin(minScore, s.score);    // ← 呼叫 template function
    if (s.score >= 60) passCount++;
    else failCount++;
}
double average = static_cast<double>(total) / students.size();
```

這段就是固定挑戰題的實際整合位置：每跑一位學生，就用 `getMax` / `getMin` 更新目前的最高分與最低分，迴圈結束後就得到全班的最高與最低分。

六、執行畫面與輸出結果

以下為實際以下列測試資料執行的真實輸出（節錄重點畫面）。測試資料為五位學生：Alice Wang(92)、Bob Chen(58)、Carol Lin(75)、David Wu(88)、Emma Hsu(45)。

6-1 新增與重複學號檢查

新增五位學生成功後，再次嘗試新增已存在的 `S001`：

```
請選擇功能：請輸入學號：[錯誤] 學號 S001 已存在，無法重複新增。
```

6-2 列出所有學生（功能 2）

```
===== 全班學生名單 =====
```

```
學號      姓名      成績
-----
S001      Alice Wang    92
S002      Bob Chen      58
S003      Carol Lin     75
S004      David Wu      88
S005      Emma Hsu      45
=====
共 5 位學生。
```

6-3 依成績排序（功能 3）

排序後成績由高到低排列（92 → 88 → 75 → 58 → 45）：

```
【成功】 已依成績由高到低排序完成。

===== 全班學生名單 =====
學號      姓名      成績
-----
S001      Alice Wang    92
S004      David Wu      88
S003      Carol Lin     75
S002      Bob Chen      58
S005      Emma Hsu      45
=====
共 5 位學生。
```

6-4 依學號查詢（功能 4）

查詢存在的 S003：

```
請輸入要查詢的學號：[找到] 學號：S003  姓名：Carol Lin  成績：75 分
```

查詢不存在的 S999：

```
請輸入要查詢的學號：[查無此人] 找不到學號為 S999 的學生。
```

6-5 成績統計（功能 5，含 template function 結果）

```
===== 成績統計結果 =====
全班人數      : 5 人
全班平均      : 71.60 分
最高分        : 92 分
最低分        : 45 分
及格人數      : 3 人 (>= 60)
不及格人數    : 2 人 (< 60)
=====
```

最高分 92、最低分 45 即由 `getMax` / `getMin` 兩個 template function 計算而來，結果與資料相符，驗證 template function 整合正確。

✦ 繳交時請依此測試流程，自行在電腦上截圖貼到報告對應位置（助教評分項目之一是「報告中的程式輸出結果需與繳

交的小專題程式一致」)。

七、GitHub Repo Link 與檔案結構說明

Repository (請改成自己的)：

```
https://github.com/your-account/your-repository
```

檔案結構：

```
your-repository/  
├─ main.cpp      # 主程式 (功能 1~5、template function 皆在此)  
├─ README.md     # 編譯與執行說明  
└─ report.pdf    # 本學習報告
```

編譯與執行：

```
g++ -std=c++17 main.cpp -o main  
./main
```

繳交前檢查清單：- [] repository 設為 **Public** (無痕視窗能打開) - [] Ecourse 繳交欄位已貼上完整 repo 網址 - [] 報告第一頁也寫了同一個 repo 網址 - [] main.cpp 能用 `g++ -std=c++17` 編譯成功

八、遇到的問題與解決方式

問題 1：輸入姓名時讀不到完整的名字

一開始我用 `cin >> s.name` 讀姓名，發現如果姓名含空白 (例如 "Alice Wang")，只會讀到 "Alice"，"Wang" 會被當成下一個輸入。

解決方式：改用 `getline(cin, s.name)` 讀整行；但要注意前一個 `cin >>` 會在緩衝區留下換行符號，所以在讀姓名前先用 `cin.ignore(numeric_limits<streamsize>::max(), '\n')` 清掉殘留的換行。

問題 2：輸入非數字導致程式無窮迴圈

當選單要求輸入數字時，如果使用者打了英文字母，`cin` 會進入錯誤狀態，之後所有讀取都失敗，造成畫面不斷重印選單。

解決方式：寫了一個 `readInt` 函式，讀取失敗時用 `cin.clear()` 清除錯誤旗標，再用 `cin.ignore` 丟掉這一整行，然後要求重新輸入，程式就不會卡死。

問題 3：最高 / 最低分的初始值該設多少

如果把 `maxScore` 初始化成 0、`minScore` 初始化成 100，遇到極端資料 (例如全班都 0 分或都 100 分) 時可能會有邏輯漏洞，而且也沒有用到題目要求的 `template function`。

解決方式：把初始值設成「第一位學生的成績」(`students[0].score`)，再用 `getMax` / `getMin` 逐筆比較。這樣既正確，又自然地把固定挑戰題的 `template function` 整合進來。

問題 4：用 map 還是 vector

我曾考慮用 `map<string, Student>`，查詢會比較快，但功能 3 要求用 `sort` 依「成績」排序，而 `map` 只能依「鍵（學號）」排序、無法依值重排。

解決方式：衡量後選擇 `vector<Student>`，讓 `sort` 能直接套用；查詢改用線性搜尋，在資料量不大時完全足夠。這個取捨也讓我更理解「依操作需求選容器」的重要性。

九、學習心得與未來可延伸方向

9-1 學習心得

這次自主學習最大的收穫，是把課本上「template 可以套用不同型別」這句抽象的話，變成我自己手上會用的工具。以前覺得 `vector`、`map` 只是「方便的東西」，現在才理解它們其實就是 `class template`，背後對應著動態陣列、紅黑樹等資料結構。寫 `getMax` / `getMin` 的過程也讓我體會到 `function template` 的好處：一份程式碼就能比較各種型別，不用為每種型別各寫一個函式。

另外，這次也讓我養成「先想清楚需要哪些操作，再決定用哪個容器」的習慣。像是為了讓排序最自然而選 `vector`，而不是看到 `map` 查詢快就盲目選 `map`，這種取捨的思考，比單純會用語法更重要。

實作過程中那幾個「卡住的小問題」（`getline` 讀姓名、`cin` 無窮迴圈）雖然當下很煩，但解決之後對 C++ 的輸入機制理解更深了，這些都是只看課本學不到的經驗。

9-2 未來可延伸方向

1. **改用 `map<string, Student>` 並比較效能：**實作另一個版本，把查詢改成 $O(\log n)$ ，體會 `vector` 與 `map` 在不同操作上的差異。
2. **加入檔案讀寫：**用 `fstream` 把學生資料存到檔案，下次開啟程式時自動載入，做到資料持久化。
3. **用 `priority_queue` 找 Top-3 高分學生：**練習優先佇列的應用。
4. **依不同欄位排序：**讓使用者選擇「依成績」或「依學號」排序，把比較規則做成可切換的 `lambda`。
5. **把 `Student` 改成 `class` 並加上封裝：**用 `getter` / `setter` 管理欄位，銜接物件導向的觀念。

透過這次練習，我對大二即將學到的「資料結構」課程更有信心了——因為我已經先用 STL「會用」這些結構，接下來只要再學它們「怎麼被實作出來」就好。